Expt. No. 1

Title: To explore python libraries for deep learning examples: Keras, TensorFlow.

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 24/7/2025

Sign:_____

Experiment No. 1

Deep Learning

Aim : To explore python libraries for deep learning eg) Keras, TensorFlow.

Software : Google Collab

Pre-Lab Questions:

1. Explain the features of Tensor flow library?
   TensorFlow is an open-source deep learning library for scalable model training with multi-device support and easy APIs.

Post-Lab Questions:

1. Explain the applications that can be implemented using Tensor Flow and Keras ?
   TensorFlow and Keras can be used to build applications like image recognition, natural language processing, and predictive analytics.

Theory : TensorFlow is an open-source framework developed by Google researchers to run machine learning, deep learning and other statistical and predictive analytics workloads. Like similar platforms, designed to streamline the process of developing and executing advanced analytics applications for users such as data scientists, statisticians and predictive modelers.

Features:

1. Prepare and load data for successful ML outcomes:

Data can be the most important factor in the success of your ML endeavors. TensorFlow offers multiple data tools to help you consolidate, clean and pre-process data at scale:

- Standard datasets for initial training and validation
- Highly scalable data pipelines for loading data
- Pre-processing layers for common input transformations
- Tools to validate and transform large datasets
- Additionally, responsible AI tools help you uncover and eliminate bias in your data to produce fair, ethical outcomes from your models.

2. Build and fine-tune models with the TensorFlow ecosystem:

TensorFlow supports distributed training, immediate model iteration and easy debugging with Keras , and much more. Tools like Model Analysis and Tensor Board help you track development and improvement through your model's lifecycle.

3. Deploy models on-device, in the browser, on-prem, or in the cloud:

TensorFlow provides robust capabilities to deployyour models on any environment - servers, edgedevices, browsers, mobile, microcontrollers, CPUs, GPUs, FPGAs. TensorFlow Serving can run ML models a

production scale on the most advanced processors in the world, including Google's custom TensorProcessing Units (TPUs). If you need to analyze data close to its source to reduce latency and improve data privacy, the TensorFlow Lite framework lets you run models on mobile devices, edge computing devices, and even microcontrollers, and the TensorFlow.js framework lets you run machine learning with just a web browser.

4. Implement MLOps for production ML: The TensorFlow platform helps you implement best practices for data automation, model tracking, performance monitoring, and model retraining. Using production-level tools to automate and track model training over the lifetime of a product, service, or business process is critical to success. TFX provides software frameworks and tooling for full MLOps deployments, detecting issues as your data and models evolve over time.

Keras

Definition: Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. It also has extensive documentation and developer guides.

Features:

1. Used for experimenting: Keras is the most used deep learning framework among top-5 winning teams on Kaggle. Because Keras makes it easier to run new experiments, it empowers you to try more ideas than your competition, faster.
2. Exascale machine learning: Built on top of TensorFlow 2, Keras is an industry-strength framework that can scale to large clusters of GPUs or an entire TPU pod. It's not only possible; is easy.
3. Deployment: Take advantage of the full deployment capabilities of the TensorFlow platform. You can export Keras models to JavaScript to run directly in the browser, to TF Lite to run on iOS, Android, and embedded devices. it is also easy to serve Keras models as via a web API.
4. Vast ecosystem:

Keras is a central part of the tightly-connected TensorFlow 2 ecosystem, covering every step of the machine learning workflow, from data management to hyperparameter training to deployment solutions.

5. State-of-the-art research:

Keras is used by CERN, NASA, NIH, and many more scientific organizations around the world (and yes, Keras is used at the LHC). Keras has the low-level flexibility to implement arbitrary research ideas while offering optional high-level convenience features to speed up experimentation cycles.

Programs & Output:

Tensorflow:

```
# importing the dependencies
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import numpy as np
import matplotlib.pyplot as plt

# Model Parameters
learning_rate = 0.01 # determines the step size at each iteration while moving toward a minimum of a loss function.
training_epochs = 2000 # An epoch is when all the training data is used at once and is defined as the total number of iteration
display_step = 200

# Training Data
train_X =
np.asarray([3.3,4.4,5.5,6.71,6.93,4.168,9.779,6.182,7.59,2.167,7.042,10.791,5.313,7.997,5.654,9.27,3.1])
train_y =
np.asarray([1.7,2.76,2.09,3.19,1.694,1.573,3.366,2.596,2.53,1.221,2.827,3.465,1.65,2.904,2.42,2.94,1.3])
n_samples = train_X.shape[0] # assigns the number of samples in the train_X dataset to the variable n_samples.

# Test Data
test_X = np.asarray([6.83, 4.668, 8.9, 7.91, 5.7, 8.7, 3.1, 2.1]) # in NumPy it is used to convert a given input (such as a list, tuple, or array-like object) into an array.
test_y = np.asarray([1.84, 2.273, 3.2, 2.831, 2.92, 3.24, 1.35, 1.03])
# Set placeholders for feature and target vectors
X = tf.placeholder(tf.float32) # sed to define a placeholder tensor that could be later fed with actual data during the execution of a computational graph.
y = tf.placeholder(tf.float32) # float32, indicates that it will hold floating-point values.

# Set model weights and bias
W = tf.Variable(np.random.randn(), name="weight")
b = tf.Variable(np.random.randn(), name="bias")

# Construct a linear model
linear_model = W*X + b
# Mean squared error
cost = tf.reduce_sum(tf.square(linear_model - y)) / (2*n_samples) # represents the mean squared error (MSE) cost function for a linear regression model.

# Gradient descent
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost) # TensorFlow sets up the computation to minimize the cost function using gradient descent.

# Initializing the variables
init = tf.global_variables_initializer() # get an operation that, when executed, initializes all the global variables in the TensorFlow graph

# Launch the graph
with tf.Session() as sess: # manager that creates a TensorFlow session and automatically manages its
```

```
execution.
# Load initialized variables in current session
 sess.run(init)

 # Fit all training data
 for epoch in range(training_epochs):
# perform gradient descent step
    sess.run(optimizer, feed_dict={X: train_X, y: train_y})

  # Display logs per epoch step
 if (epoch+1) % display_step == 0:
  c = sess.run(cost, feed_dict={X: train_X, y: train_y})

 print("Epoch:{0:6} \t Cost:{1:10.4} \t W:{2:6.4} \t b:{3:6.4}".format(epoch+1, c, sess.run(W), sess.run(b)))
 # Print final parameter values
 print("Optimization Finished!")

 training_cost = sess.run(cost, feed_dict={X: train_X, y: train_y})
 print("Final training cost:", training_cost, "W:", sess.run(W), "b:", sess.run(b), '\n')

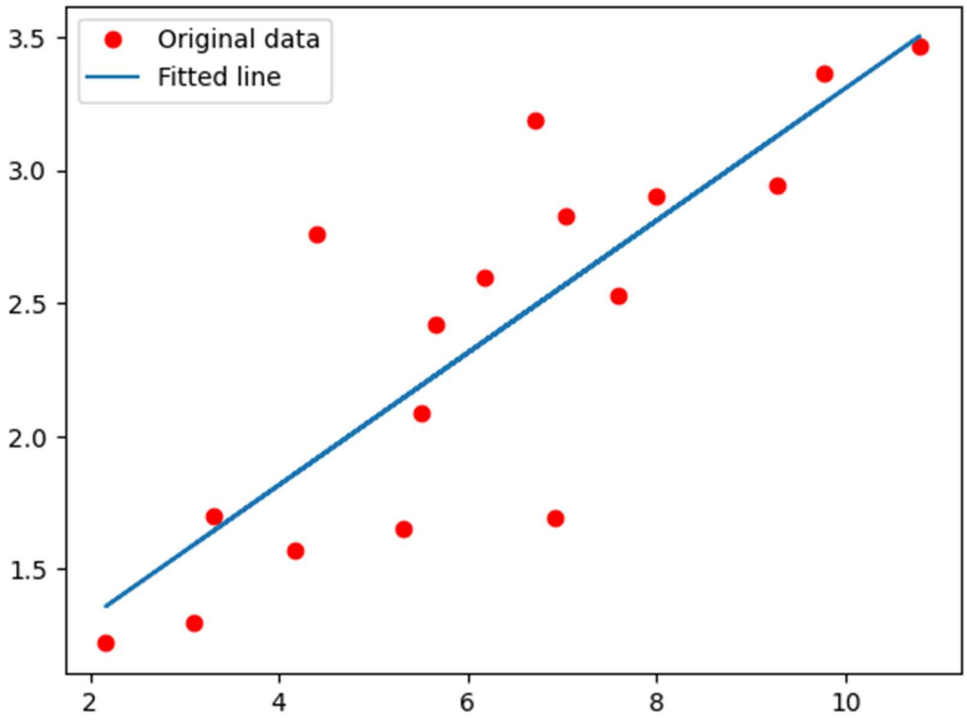
 # Graphic display
 plt.plot(train_X, train_y, 'ro', label='Original data')
 plt.plot(train_X, sess.run(W) * train_X + sess.run(b), label='Fitted line')
 plt.legend()
 plt.show()

 # Testing the model
 testing_cost = sess.run(tf.reduce_sum(tf.square(linear_model - y)) / (2 * test_X.shape[0]), feed_dict={X:
test_X, y: test_y})
 print("Final testing cost:", testing_cost)
 print("Absolute mean square loss difference:", abs(training_cost - testing_cost))

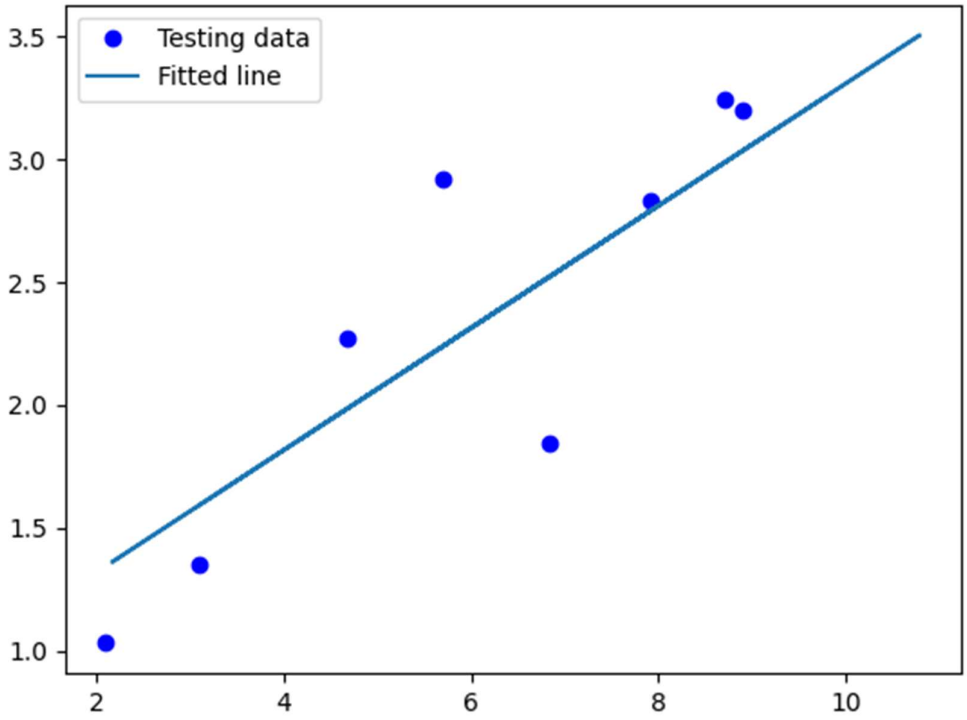
# Display fitted line on test data
 plt.plot(test_X, test_y, 'bo', label='Testing data')
 plt.plot(train_X, sess.run(W) * train_X + sess.run(b), label='Fitted line')
 plt.legend()
 plt.show()
```

OUTPUT:

```
Instructions for updating:
non-resource variables are not supported in the long term
Epoch:  2000    Cost:   0.07696           W:0.2485        b:0.8209
Optimization Finished!
Final training cost: 0.076958984 W: 0.24852164 b: 0.820873
```



Final testing cost: 0.07895341
Absolute mean square loss difference: 0.0019944236



PyTorch:

PyTorch
import torch # developing and training neural network based deep learning models.
import math # extends the list of mathematical functions.
dtype = torch.float

```
device = torch.device("cpu")
# device = torch.device("cuda:0") # Uncomment this to run on GPU

# Create random input and output data
x = torch.linspace(-math.pi, math.pi, 2000, device=device, dtype=dtype)
y = torch.sin(x)

# Randomly initialize weights
a = torch.randn((), device=device, dtype=dtype)
b = torch.randn((), device=device, dtype=dtype)
c = torch.randn((), device=device, dtype=dtype)
d = torch.randn((), device=device, dtype=dtype)
learning_rate = 1e-6

for t in range(2000):
# Forward pass: compute predicted y
  y_pred = a + b * x + c * x ** 2 + d * x ** 3
# Compute and print loss
loss = (y_pred - y).pow(2).sum().item()

if t % 100 == 99:
 print(t, loss)

# Backprop to compute gradients of a, b, c, d with respect to loss
grad_y_pred = 2.0 * (y_pred - y)
grad_a = grad_y_pred.sum()
grad_b = (grad_y_pred * x).sum()
grad_c = (grad_y_pred * x ** 2).sum() # calculates the gradient of some scalar variable c
with respect to the tensor x in PyTorch.
grad_d = (grad_y_pred * x ** 3).sum()

# Update weights using gradient descent
a -= learning_rate * grad_a # updates the value of some scalar variable a using gradient
descent in PyTorch.
b -= learning_rate * grad_b
c -= learning_rate * grad_c
d -= learning_rate * grad_d
print(f'Result: y = {a.item()} + {b.item()} x + {c.item()} x^2 + {d.item()} x^3')
```

OUTPUT:

```
1999 6153.05859375
Result: y = -0.8028205037117004 + 1.1849738359451294 x + 0.47501543164253235 x^2 +
-0.15010419487953186 x^3
```

Conclusion:

Exploring Python libraries like TensorFlow and Keras enables efficient development and deployment of deep learning models with powerful, user-friendly tools. These libraries simplify complex neural network design, making advanced AI accessible for a wide range of applications.

CO's Covered:

CO1. Implement basic neural network models.